

# N-gram Language Models

CS221.Q21.KHTN - Xử lý ngôn ngữ tự nhiên

Bùi Ngọc Thiên Thanh - 23521436

Cáp Kim Hải Anh - 23520036

Hoàng Đức Dũng - 23520328

Nguyễn Thái Sơn - 23521356



Khoa Khoa học Máy tính  
Trường Đại học Công nghệ thông tin  
VNU-HCM

Tháng 2, 2026

# Tables of contents

- ➊ Giới thiệu mô hình Ngôn ngữ N-gram (N-gram Language Models)
  - Động lực: Dự đoán từ tiếp theo (Next-word Prediction)
  - Khái niệm N-gram (N-gram Concept)
- ➋ 3.1.N-grams: Xác suất từ theo ngữ cảnh
- ➌ 3.1.1 Giả định Markov
  - Khái niệm & Công thức
  - Mô hình Bigram & N-gram
- ➍ 3.1.2. Ước lượng xác suất trong mô hình N-gram
- ➎ 3.1.3 Xử lý vấn đề quy mô trong N-gram lớn



- Con người có khả năng tự nhiên dự đoán từ tiếp theo trong câu.
- Hãy thử đoán từ còn thiếu trong câu sau:

*“Mặt nước của hồ Hoàn Kiếm trông thật \_\_\_\_\_”*

Bạn chọn đáp án nào?

- A. trong xanh
- B. yên bình
- C. tử lạnh
- D. ngẫu nhiên



- Con người có khả năng tự nhiên dự đoán từ tiếp theo trong câu.
- Hãy thử đoán từ còn thiếu trong câu sau:

*“Mặt nước của hồ Hoàn Kiếm trông thật \_\_\_\_\_”*

Bạn chọn đáp án nào?

- A. trong xanh
- B. yên bình
- C. tử lạnh
- D. ngẫu nhiên

- Hầu hết mọi người sẽ chọn A hoặc B.
- ⇒ Ngôn ngữ không hoàn toàn ngẫu nhiên mà tuân theo ngữ cảnh và quy luật.



# Language Model (Mô hình ngôn ngữ) là gì?

- **Language Model (LM)** là mô hình dùng để:
  - Dự đoán từ tiếp theo trong câu
  - Đánh giá mức độ tự nhiên của một chuỗi từ
- Mô hình học các mẫu (patterns) trong ngôn ngữ từ dữ liệu văn bản lớn.
- Ví dụ:

## Câu tự nhiên

đột nhiên tôi thấy ba người đang đứng trên vỉa hè

## Câu kém tự nhiên

đứng người ba tôi trên thấy hè vỉa đang đột nhiên



# Vai trò của việc dự đoán từ

- Nhiều hệ thống AI hiện đại hoạt động dựa trên:

## Next-word prediction (dự đoán từ tiếp theo)

- Ứng dụng:
  - Kiểm tra ngữ pháp (Grammar correction)
  - Nhận dạng giọng nói (Speech recognition)
  - Gợi ý khi nhập văn bản (Text autocomplete)
  - Hệ thống hỗ trợ giao tiếp AAC (Augmentative and Alternative Communication)
- Ví dụ: mô hình chọn câu có xác suất tự nhiên cao hơn
  - There are** two exams > Their are two exams
  - Everything has **improved** > Everything has improve



# N-gram là gì? (What is an N-gram?)

- **N-gram** là chuỗi gồm  $n$  từ xuất hiện liên tiếp trong văn bản.
- Đây là cách đơn giản để biểu diễn ngữ cảnh (context) của từ trong câu.
- Ví dụ:
  - Unigram ( $n = 1$ ): Mặt
  - Bigram ( $n = 2$ ): Mặt nước
  - Trigram ( $n = 3$ ): Mặt nước của



# Ý tưởng chính của N-gram

- Khi con người hiểu ngôn ngữ, các từ gần nhau thường có liên hệ mạnh.
- N-gram mô tả câu bằng các nhóm từ liên tiếp thay vì toàn bộ câu.
- Ví dụ câu:

Mặt nước của hồ Hoàn Kiếm

- Bigram:

Mặt nước → nước của → của hồ → hồ Hoàn → Hoàn Kiếm

- Trigram:

Mặt nước của → nước của hồ → của hồ Hoàn → hồ Hoàn Kiếm



# Vì sao sử dụng N-gram?

- Giúp mô hình hoá ngôn ngữ theo cách đơn giản và trực quan.
- Cho phép máy tính học các mẫu xuất hiện phổ biến của từ.
- Là bước khởi đầu để xây dựng các Language Model phức tạp hơn.

## Ý tưởng cốt lõi

Ngữ cảnh gần thường mang nhiều thông tin quan trọng cho việc hiểu ngôn ngữ.



# Bài toán cơ bản (Core Problem)

- Mục tiêu: tính xác suất của một từ  $w$  dựa trên lịch sử (history)  $h$ :

$$P(w|h)$$

- Ví dụ:

History: *"Mặt nước của hồ Hoàn Kiếm trông thật"*

- Ta muốn tính:

$$P(\text{xanh}|\text{Mặt nước của hồ Hoàn Kiếm trông thật})$$



# Ước lượng bằng tần suất (Relative Frequency Estimation)

- Một cách đơn giản:
  - Dùng corpus (tập văn bản lớn)
  - Đếm số lần xuất hiện

- Ý tưởng:

*Trong tất cả các lần xuất hiện history  $h$ , bao nhiêu lần được theo sau bởi từ  $w$ ?*

$$P(w|h) = \frac{C(h, w)}{C(h)}$$



## Công thức

$$P(\text{xanh} \mid \text{Mặt nước của hồ Hoàn Kiếm trông thật}) \\ = \frac{C(\text{Mặt nước của hồ Hoàn Kiếm trông thật xanh})}{C(\text{Mặt nước của hồ Hoàn Kiếm trông thật})}$$

- $C(\cdot)$ : số lần xuất hiện (Count)
- Ước lượng dựa trên thống kê thực nghiệm (empirical statistics)



# Vấn đề của phương pháp đếm trực tiếp

- Ngôn ngữ có tính sáng tạo (Language is creative).
- Rất nhiều câu:
  - chưa từng xuất hiện trong corpus
  - hoặc xuất hiện cực kỳ hiếm
- Ngay cả toàn bộ Internet cũng:
  - Không đủ dữ liệu cho mọi câu hoàn chỉnh.
- $\Rightarrow$  Không thể ước lượng xác suất từ toàn bộ lịch sử dài.



# Chain Rule of Probability (Quy tắc dây chuyền)

- Ta muốn tính xác suất của cả một chuỗi:

$$P(X_1, X_2, \dots, X_n)$$

- Chain Rule cho phép phân rã xác suất này thành các xác suất có điều kiện:

## Chain Rule

$$P(X_1 \dots X_n) = \prod_{k=1}^n P(X_k | X_{1:k-1})$$

- Nghĩa là: mỗi phần tử được dự đoán dựa trên toàn bộ lịch sử trước đó.



# Chain Rule trong Language Modeling

- Khi áp dụng cho ngôn ngữ:

$$P(w_{1:n}) = \prod_{k=1}^n P(w_k | w_{1:k-1})$$

- Xác suất của cả câu được tạo bằng cách:

**dự đoán từng từ dựa trên các từ trước đó**

- Ví dụ:

$$P(w_1) P(w_2 | w_1) P(w_3 | w_{1:2}) \dots P(w_n | w_{1:n-1})$$

- Đây là nguyên lý cốt lõi của:
  - Language Models
  - Large Language Models (LLMs)



# Vấn đề còn tồn tại

- Trong thực tế:
  - Toàn bộ lịch sử  $w_{1:n-1}$  có thể rất dài
  - Số lượng ngữ cảnh tăng theo cấp số nhân
  - Nhiều chuỗi chưa từng xuất hiện trong corpus
- Ngôn ngữ có tính **sáng tạo** (language is creative).
- $\Rightarrow$  Không thể ước lượng chính xác bằng cách đếm trực tiếp.

**Chain rule đúng về mặt toán học, nhưng khó áp dụng trong thực tế.**



# Ý tưởng dẫn đến N-gram

## Key Insight

Thay vì sử dụng toàn bộ lịch sử dài, ta chỉ xét một phần ngữ cảnh gần nhất.

- Mục tiêu:
  - Đơn giản hóa việc ước lượng xác suất
  - Giảm số lượng ngữ cảnh cần quan sát
- Ý tưởng này dẫn đến:

## Mô hình N-gram



# Giả định Markov là gì?

**Ý tưởng:** Tính xác suất từ tiếp theo dựa vào toàn bộ từ trước đó

$\Rightarrow$

Tính xác suất từ tiếp theo dựa vào một **vài từ gần nhất trước đó**.



# Công thức xấp xỉ (Approximation)

Thay vì tính toán trên toàn bộ chuỗi  $w_{1:n-1}$ :

$$P(w_n | w_1, w_2, \dots, w_{n-1})$$

Ta sử dụng phép xấp xỉ Markov:

$$\approx P(w_n | w_{n-k}, \dots, w_{n-1})$$

- Chỉ quan tâm đến  $k$  từ đứng ngay trước đó.



# Mô hình Bigram (2-gram)

Trường hợp đơn giản nhất: Chỉ nhìn lại **1 từ** ( $N - 1$ ).

## Công thức Bigram

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-1})$$

**Ví dụ:** *"Tôi muốn ăn bánh"*

- **Thực tế:**  $P(\text{bánh} | \text{Tôi muốn ăn})$
- **Bigram:**  $P(\text{bánh} | \text{ăn})$
- $\rightarrow$  Chỉ cần biết từ "ăn" để đoán "bánh".



# Tổng quát hóa N-gram

- **Bigram** ( $N = 2$ ): Nhìn lại 1 từ.
- **Trigram** ( $N = 3$ ): Nhìn lại 2 từ.
- **4-gram** ( $N = 4$ ): Nhìn lại 3 từ.
- **N-gram**: Nhìn lại  $N - 1$  từ phía trước.

## Công thức N-gram

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-N+1:n-1})$$

**Ví dụ:** "Vui lòng không hút thuốc" (Dự đoán từ "thuốc")

| Mô hình | Ngữ cảnh sử dụng | Xấp xỉ                               |
|---------|------------------|--------------------------------------|
| Bigram  | hút              | $P(\text{thuốc}   \text{hút})$       |
| Trigram | không hút        | $P(\text{thuốc}   \text{không hút})$ |



# Phương pháp ước lượng hợp lý cực đại - MLE

- **Vấn đề:** Làm thế nào để tính toán xác suất của một n-gram (ví dụ: xác suất từ tiếp theo dựa vào từ trước đó)?
- **Giải pháp:** Sử dụng Ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE).
- **Cách hoạt động:**
  - Trích xuất tần suất xuất hiện của các cụm từ từ một tập dữ liệu văn bản lớn (corpus).
  - Chuẩn hóa (normalize) các số đếm này sao cho kết quả là xác suất nằm trong khoảng từ 0 đến 1 (và tổng các xác suất bằng 1).
- **Ý nghĩa cốt lõi:** MLE tìm ra bộ tham số giúp tối đa hóa khả năng xảy ra của tập huấn luyện dựa trên mô hình đã cho ( $P(T|M)$ ).



# Công thức toán học cho Bigram (2-gram)

- Để tính xác suất của một từ  $w_n$  khi biết từ đứng ngay trước nó là  $w_{n-1}$ , ta chia số lần xuất hiện của cả cụm (bigram) cho tổng số lần xuất hiện của mọi cụm bắt đầu bằng từ  $w_{n-1}$ :
- Công thức:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

- Công thức rút gọn: Tổng số lượng mọi bigram bắt đầu bằng  $w_{n-1}$  chính là số lần xuất hiện của từ  $w_{n-1}$  (unigram count).

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

với  $C$  là tần suất xuất hiện.



# Tổng quát hóa cho mô hình N-gram

- Nguyên lý của Bigram có thể mở rộng cho cấu trúc N-gram bất kỳ (3-gram, 4-gram,...).
- Công thức tổng quát: Ước lượng xác suất bằng cách chia tần suất của toàn bộ chuỗi cho tần suất của phần tiền tố (prefix) của chuỗi đó.

$$P(w_n | w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1} w_n)}{C(w_{n-N+1:n-1})}$$

- Tỷ số này được gọi là Tần suất tương đối (Relative Frequency).



# Khảo sát với một Corpus nhỏ

- Chuẩn bị dữ liệu: Cần thêm ký hiệu đặc biệt `<s>` (bắt đầu) và `</s>` (kết thúc) vào mỗi câu.
- Mục đích: Cung cấp ngữ cảnh bigram cho từ đầu tiên và đảm bảo tổng xác suất của các câu tạo thành một phân phối chuẩn hợp lệ.
- Corpus ví dụ (3 câu):
  - 1. `<s> I am Sam </s>`
  - 2. `<s> Sam I am </s>`
  - 3. `<s> I do not like green eggs and ham </s>`



# Tính xác suất Bigram cụ thể

- Xác suất từ đầu câu (dựa vào  $\langle s \rangle$  xuất hiện 3 lần):  
 $P(I | \langle s \rangle) = \frac{2}{3} = 0.67$  (Từ "I" đứng đầu 2/3 câu)  
 $P(Sam | \langle s \rangle) = \frac{1}{3} = 0.33$  (Từ "Sam" đứng đầu 1/3 câu)
- Xác suất các từ tiếp theo (dựa vào từ "I" xuất hiện 3 lần):  
 $P(am | I) = \frac{2}{3} = 0.67$  (Sau "I" là "am" 2 lần)  
 $P(do | I) = \frac{1}{3} = 0.33$  (Sau "I" là "do" 1 lần)
- Xác suất kết thúc câu:  
 $P(\langle /s \rangle | Sam) = \frac{1}{2} = 0.5$  (Sau "Sam" là kết thúc câu 1 trên 2 lần)



# Ví dụ thực tế - Dự án nhà hàng Berkeley

- Dữ liệu thực tế từ hệ thống hỏi đáp nhà hàng ở Berkeley (9.332 câu).
- Bảng minh họa đếm số lần từ ở "Cột" xuất hiện ngay sau từ ở "Hàng":

|                | <b>i</b> | <b>want</b> | <b>to</b> | <b>eat</b> | <b>chinese</b> | <b>food</b> | <b>lunch</b> | <b>spend</b> |
|----------------|----------|-------------|-----------|------------|----------------|-------------|--------------|--------------|
| <b>i</b>       | 5        | 827         | 0         | 9          | 0              | 0           | 0            | 2            |
| <b>want</b>    | 2        | 0           | 608       | 1          | 6              | 6           | 5            | 1            |
| <b>to</b>      | 2        | 0           | 4         | 686        | 2              | 0           | 6            | 211          |
| <b>eat</b>     | 0        | 0           | 2         | 0          | 16             | 2           | 42           | 0            |
| <b>chinese</b> | 1        | 0           | 0         | 0          | 0              | 82          | 1            | 0            |
| <b>food</b>    | 15       | 0           | 15        | 0          | 1              | 4           | 0            | 0            |
| <b>lunch</b>   | 2        | 0           | 0         | 0          | 0              | 1           | 0            | 0            |
| <b>spend</b>   | 1        | 0           | 1         | 0          | 0              | 0           | 0            | 0            |



# Ví dụ thực tế - Dự án nhà hàng Berkeley

- Bảng Xác suất Bigram (Bigram Probabilities): Được tính bằng cách lấy dữ liệu đếm chia cho số lần xuất hiện của từ đứng trước (Unigram count: I=2533, want=927, to=2417, eat=746...):

|                | <b>i</b> | <b>want</b> | <b>to</b> | <b>eat</b> | <b>chinese</b> | <b>food</b> | <b>lunch</b> | <b>spend</b> |
|----------------|----------|-------------|-----------|------------|----------------|-------------|--------------|--------------|
| <b>i</b>       | 0.002    | 0.33        | 0         | 0.0036     | 0              | 0           | 0            | 0.00079      |
| <b>want</b>    | 0.0022   | 0           | 0.66      | 0.0011     | 0.0065         | 0.0065      | 0.0054       | 0.0011       |
| <b>to</b>      | 0.00083  | 0           | 0.0017    | 0.28       | 0.00083        | 0           | 0.0025       | 0.087        |
| <b>eat</b>     | 0        | 0           | 0.0027    | 0          | 0.021          | 0.0027      | 0.056        | 0            |
| <b>chinese</b> | 0.0063   | 0           | 0         | 0          | 0              | 0.52        | 0.0063       | 0            |
| <b>food</b>    | 0.014    | 0           | 0.014     | 0          | 0.00092        | 0.0037      | 0            | 0            |
| <b>lunch</b>   | 0.0059   | 0           | 0         | 0          | 0              | 0.0029      | 0            | 0            |
| <b>spend</b>   | 0.0036   | 0           | 0.0036    | 0          | 0              | 0           | 0            | 0            |



- Áp dụng Bigram để tính xác suất cho một câu hoàn chỉnh:
  - Ví dụ: Xác suất để câu "I want english food" xuất hiện là bao nhiêu?
  - Áp dụng luật chuỗi (Chain Rule) bằng xác suất Bigram:

$$\begin{aligned}P(< s > \text{ i want english food } < /s >) &= P(i | < s >) \times P(want | i) \\&\times P(english | want) \times P(food | english) \times P(< /s > | food) \\&= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 = 0.000031\end{aligned}$$



# Thống kê Bigram phản ánh điều gì?

- Các xác suất Bigram không chỉ là những con số vô hồn, chúng đã tự động "học" và mã hóa các tri thức về ngôn ngữ:
- Sự thật về Cú pháp: Ví dụ sau "eat" thường là danh từ/tính từ, sau "to" thường là động từ.
- Đặc thù ngữ cảnh tác vụ: Trong ngữ cảnh hệ thống trợ lý ảo, xác suất câu bắt đầu bằng từ đại từ "I" là rất lớn.
- Hiện tượng văn hóa: Xác suất người dùng tìm kiếm cụm "Chinese food" cao hơn đáng kể so với "English food" dựa trên thói quen người dùng tại vùng Berkeley.



# Vấn đề khi N-gram có quy mô lớn

Trong thực tế, mô hình N-gram có thể rất lớn, gây ra:

- Xác suất cực nhỏ  $\rightarrow$  tràn số dưới (underflow)
- Số lượng n-gram tăng theo cấp số mũ
- Bộ nhớ lưu trữ rất lớn
- Tính toán chậm

*Bài toán không còn chỉ là NLP mà là bài toán hệ thống.*



# Vấn đề 1: Tràn số dưới (Numerical Underflow)

Xác suất câu được tính bằng:

$$P = p_1 \times p_2 \times p_3 \times \dots$$

Vì  $p_i \leq 1$ , khi nhân nhiều xác suất nhỏ:

$$0.01 \times 0.02 \times 0.005 \rightarrow 10^{-9}$$

Nếu nhân nhiều hơn nữa:

$$\rightarrow 10^{-40} \approx 0$$

**Máy tính làm tròn về 0  $\Rightarrow$  mất thông tin.**



# Giải pháp: Tính trong không gian log

Thay vì nhân xác suất:

$$p_1 \times p_2 \times p_3$$

Ta chuyển sang:

$$\log p_1 + \log p_2 + \log p_3$$

Khôi phục lại xác suất khi cần:

$$P = \exp(\log P)$$

- Tránh underflow
- Ổn định số học
- Tính toán hiệu quả hơn



# Ngữ cảnh dài hơn (Higher-order N-grams)

Khi có nhiều dữ liệu hơn, ta dùng:

- Bigram ( $n=2$ )
- Trigram ( $n=3$ )
- 4-gram, 5-gram

Ví dụ:

$$P(w_n \mid w_{n-2}, w_{n-1})$$

Nếu kích thước từ vựng là  $V$ :

Số khả năng n-gram =  $V^n$

**Tăng theo cấp số mũ.**



# Dữ liệu N-gram quy mô rất lớn

Một số tập dữ liệu lớn:

- Google Web 5-gram (1 nghìn tỷ từ)
- Google Books N-grams (800 tỷ token)
- COCA (1 tỷ từ)

Lưu trữ toàn bộ bảng n-gram:

⇒ Tốn bộ nhớ khổng lồ



# Các kỹ thuật tối ưu hệ thống

- **1. Lượng tử hóa (Quantization):** Dùng 4–8 bit thay vì số thực 8 byte
- **2. Hash 64-bit:** Không lưu chuỗi từ đầy đủ trong RAM
- **3. Reverse Trie:** Tra cứu n-gram hiệu quả
- **4. Pruning:** Loại bỏ n-gram hiếm



# Infini-gram ( $\infty$ -gram)

- Ý tưởng:
  - Không tiền tính toán toàn bộ bảng n-gram
  - Tính xác suất tại thời điểm suy luận
  - Sử dụng cấu trúc dữ liệu **suffix array**
- Cho phép sử dụng ngữ cảnh dài tùy ý, ngay cả với corpus hàng nghìn tỷ token.



| Vấn đề | Giải pháp |
|--------|-----------|
|--------|-----------|

|                  |              |
|------------------|--------------|
| Underflow        | Log-space    |
| Bộ nhớ lớn       | Quantization |
| Quá nhiều n-gram | Pruning      |
| Ngữ cảnh dài     | Infini-gram  |

*Mở rộng N-gram là bài toán kỹ thuật và tối ưu hệ thống.*



# Câu hỏi thảo luận (Q&A)

- **1:** Vì sao khi tính xác suất câu bằng  $N$  – *gram* ta dễ gặp hiện tượng underflow?
- **2:** Ý nghĩa của các ký hiệu đặc biệt  $\langle s \rangle$  và  $\langle /s \rangle$  trong việc xây dựng mô hình ngôn ngữ là gì?
- **3:** Sự khác biệt mấu chốt giữa mô hình  $n$ -gram truyền thống và **Infini-gram** ( $\infty$ -gram) trong việc lưu trữ dữ liệu là gì?
- **4:** Nếu  $C(\text{want}) = 927$  và  $C(\text{want to}) = 608$ , hãy thiết lập công thức tính  $P(\text{to}|\text{want})$  theo phương pháp MLE?



- 1. Vì phải nhân rất nhiều xác suất nhỏ hơn 1.
- 2.  $\langle s \rangle$  tạo ngữ cảnh cho từ bắt đầu câu;  $\langle /s \rangle$  giúp mô hình tính toán xác suất kết thúc câu, tạo thành một phân phối xác suất chuẩn.
- 3.  $n$ -gram truyền thống lưu **bảng tần suất** (giới hạn  $n$ ), còn Infini-gram lưu **văn bản gốc + mảng hậu tốt** (cho phép  $n$  vô hạn).
- 4. Công thức MLE:

$$P(\text{to}|\text{want}) = \frac{C(\text{want to})}{C(\text{want})} = \frac{608}{927} \approx 0.66$$

